

Guia educativo — Next.js e a seção CIPA do SmartCampus

Documento didático para entender **o que estamos construindo** e **como o Next.js funciona**. Escrito em linguagem simples, para quem está aprendendo. Local deste arquivo: `apps/web/app/cipa/GUIA-NEXTJS-CIPA.md` (é só um `.md` — o Next.js ignora; não vira página nem atrapalha o site).

Índice

1. O panorama: React, Next.js e por quê
 2. O formato deste projeto (monorepo)
 3. Roteamento: pastas viram endereços
 4. Componentes e JSX
 5. Server Components vs Client Components
 6. Layouts e `metadata`
 7. Navegação com `Link`
 8. Buscando dados (data fetching)
 9. Estilo com Tailwind CSS
 10. As páginas da CIPA explicadas
 11. Detalhes do Next.js 16 (pegadinhas)
 12. Glossário rápido
-

1. O panorama

- **React** é uma biblioteca para construir **interfaces** (telas) montando "peças" reutilizáveis chamadas **componentes**. Cada componente é uma **função** que devolve algo parecido com HTML.
- **Next.js** é um **framework** construído em cima do React. Ele adiciona o que falta para fazer um site/app completo: sistema de **endereços (rotas)**, execução no **servidor**, otimização de imagens, build de produção, etc.
- **Por que usar Next em vez de React puro?** Porque ele já resolve roteamento, renderização no servidor (páginas mais rápidas e melhores para busca) e organização do projeto, sem você ter que montar tudo na mão.

Versões aqui: **Next.js 16.2**, **React 19**.

2. O formato deste projeto (monorepo)

O repositório é um **monorepo** (vários projetos relacionados na mesma pasta), organizado com a ferramenta **Turborepo**:

```
nextjs-google-maps/
├─ apps/
│   ├─ web/    → o SITE (Next.js)    ← é aqui que trabalhamos
│   └─ api/    → o SERVIDOR de dados (Bun + ElysiaJS, lê sensores, gera PDF)
└─ packages/  → código compartilhado (ex.: @smartcampus/types)
```

- **apps/web** é a parte visual (front-end). Rotas atuais: `maps`, `reports` e a nossa nova `cipa`.
- **apps/api** é o back-end. Ele expõe dados em `http://localhost:3001` e o site consome esses dados.
- **Runtime:** o projeto roda com **Bun** (um substituto do Node.js). Pacotes do site: Next, React, Tailwind v4, shadcn/ui, Radix, lucide-react (ícones).

3. Roteamento: pastas viram endereços

Esta versão do Next usa o **"App Router"**. A regra de ouro:

A estrutura de pastas dentro de `app/` É o mapa de URLs do site. Uma pasta com um arquivo `page.tsx` vira uma página acessível.

Arquivos com **nomes especiais** dentro de uma pasta de rota:

Arquivo	Para que serve
<code>page.tsx</code>	O conteúdo daquela URL (a página em si).
<code>layout.tsx</code>	Uma "moldura" que envolve a página e suas subpáginas.
<code>loading.tsx</code>	Tela de carregamento (opcional).
<code>error.tsx</code>	Tela de erro (opcional).

Exemplo com a nossa CIPA:

```
app/cipa/
├─ layout.tsx      → envolve tudo que começa com /cipa
├─ page.tsx        → URL: /cipa
├─ work/page.tsx   → URL: /cipa/work
└─ public/page.tsx → URL: /cipa/public
```

Ou seja: **criar a pasta** `work/` **com um** `page.tsx` **já cria o endereço** `/cipa/work` — não precisa "registrar" rota em lugar nenhum.

4. Componentes e JSX

Um **componente** é uma função que devolve **JSX**. JSX é uma sintaxe que parece HTML, mas vive dentro do código:

```
export default function CipaPage() {
  return (
    <main>
      <h1>CIPA</h1>
    </main>
  );
}
```

Pontos de atenção do JSX (diferenças do HTML puro):

- Usa-se `className` no lugar de `class`.
- Para inserir **código/variáveis** no meio do JSX, use **chaves** `{ }`: `{children}`, `{sensor.name}`.
- `export default` marca o **item principal** do arquivo (a página/componente que o Next vai usar).

5. Server Components vs Client Components

O **conceito mais importante do Next moderno**. Todo componente roda em um de dois lugares:

Server Component (é o padrão)

- Roda **no servidor**, antes de chegar ao navegador. Entrega HTML pronto.
- **Pode:** acessar banco de dados, ler arquivos, usar segredos (chaves de API), e fazer `await fetch( ... )` direto.
- **Não pode:** interatividade de navegador — nada de `useState`, `onClick`, `useEffect`, `window`, `localStorage`.
- As **4 páginas da CIPA são Server Components** (nenhuma tem `"use client"`).

Client Component

- Criado colocando `"use client"` na **primeira linha** do arquivo.
- Roda **no navegador** do usuário.
- **Pode:** ser interativo — `useState`, `onClick`, `useEffect`, etc.
- Exemplo: o antigo `PDFGenerator.tsx` era Client (`"use client"`) porque tinha botão com `onClick`, guardava a data com `useState` e usava `useEffect`.

Por que existe essa divisão?

1. **Velocidade** — quanto mais o servidor entrega pronto, menos JavaScript o navegador baixa → site abre mais rápido.
2. **Segurança** — segredos ficam só no servidor; um Client Component vai inteiro para o navegador, então nunca deve conter senha/chave.

Regra prática

Comece sempre com **Server Component**. Só use `"use client"` **quando precisar de interação**, e apenas no **pedaço interativo** (não na página toda).

Quando criarmos os **formulários** de registro de acidentes, aí usaremos Client Components.

6. Layouts e `metadata`

Um `layout.tsx` é uma moldura compartilhada. Ele recebe um `children` (o conteúdo da página atual) e o coloca dentro de uma estrutura comum:

```
export default function CipaLayout({ children }: { children: React.ReactNode }) {
  return <div className="flex flex-col h-screen">{children}</div>;
}
```

- `children` = "a página que está sendo aberta agora". O layout desenha em volta e injeta a página ali.
- Layouts **não recarregam** ao navegar entre subpáginas — só o `children` troca. Ótimo para menus/cabeçalhos fixos (que ainda podemos adicionar à CIPA).

`metadata` são as informações da aba do navegador:

```
export const metadata: Metadata = {
  title: "CIPA | SmartCampus Mauá",
  description: "Comissão Interna de Prevenção de Acidentes",
};
```

O Next lê esse objeto automaticamente e coloca o `title` na aba.

7. Navegação com `Link`

Para ir de uma página a outra **dentro do site**, usamos o componente `Link` do Next (em vez do `<a>` comum):

```
import Link from "next/link";

<Link href="/cipa/work">Acidentes de Trabalho</Link>
```

Vantagem: o `Link` troca de página **sem recarregar o site inteiro** (navegação instantânea) e ainda pré-carrega a página de destino. Use `<a>` apenas para links **externos** (outros sites).

8. Buscando dados (data fetching)

Como as páginas (Server Components) rodam no servidor, elas podem buscar dados **direto**, com `await`. Neste projeto, as funções de busca ficam em `apps/web/lib/api.ts`. Exemplo real:

```
const API_URL = process.env.NEXT_PUBLIC_API_URL ?? "http://localhost:3001";

export async function fetchRestaurants(): Promise<SensorsResponse> {
  const res = await fetch(`${API_URL}/restaurant/sensors`, {
    next: { revalidate: 30 }, // guarda o resultado por 30s antes de buscar de novo
  });
  if (!res.ok) throw new Error(`Failed: ${res.statusText}`);
  return res.json();
}
```

E uma página usa assim:

```
export default async function Page() { // repare no "async"
  const dados = (await fetchRestaurants()).data;
  return <div>{/* ... usa os dados ... */}</div>;
}
```

- Como o `fetch` acontece **no servidor**, **não há problema de CORS** (aquele erro de "site A não pode chamar site B" que ocorre no navegador).
- `next: { revalidate: 30 }` controla o cache: reaproveita a resposta por 30 segundos. (Sem isso, no Next 16 o `fetch` **não** guarda cache por padrão.)
- Nossas páginas de CIPA **ainda não buscam dados** — são só layout. Quando formos plugar dados reais, seguiremos esse mesmo padrão.

9. Estilo com Tailwind CSS

Tailwind é "utility-first": em vez de escrever CSS num arquivo separado, você monta o visual juntando classinhas prontas direto no elemento. Cada classe = uma regrinha de CSS:

Classe	Significado
<code>flex</code> / <code>flex-col</code>	layout flexível / em coluna
<code>items-center</code> / <code>justify-center</code>	centraliza na horizontal / vertical
<code>p-8</code>	espaço interno de 32px (padding)
<code>gap-6</code>	24px de espaço entre os filhos
<code>mb-2</code>	margem inferior de 8px
<code>text-4xl</code> / <code>font-bold</code>	fonte grande / negrito
<code>text-gray-600</code>	cor cinza médio
<code>rounded-xl</code> / <code>border</code>	cantos arredondados / borda
<code>bg-green-50</code>	fundo verde bem claro

Escala: os números são múltiplos de **4px** (`p-8` = $8 \times 4 = 32\text{px}$).

Prefixos (quando aplicar):

- `hover:bg-green-100` → muda só **ao passar o mouse**.
- `sm:grid-cols-2` → 2 colunas só **em telas ≥ pequenas** (responsividade).
- `group-hover:translate-x-1` → aplica quando o **card pai** (marcado `group`) está sob o mouse — por isso a setinha desliza.

Como está montado neste projeto (Tailwind v4 + shadcn):

- O `app/globals.css` liga o Tailwind com uma linha: `@import "tailwindcss";`.
- Ele define **cores nomeadas** em `:root` (tema claro) e `.dark` (tema escuro) — ex. `--primary`, `--card`, `--border`. Daí vêm classes como `bg-background` e `text-foreground`, que trocam sozinhas no modo escuro. As cores usam `oklch()`.
- **shadcn/ui** (em `components/ui/`) são componentes prontos (botão, calendário, popover) feitos sobre **Radix** + Tailwind, com o código dentro do próprio projeto (você pode editar).

10. As páginas da CIPA explicadas

`cipa/page.tsx` — **página inicial** (`/cipa`)

Server Component. Mostra um cabeçalho ("CIPA" + subtítulo) e uma **grade com 2 cards**, cada um sendo um `Link` :

- Card verde 🚒 → `/cipa/work` (Acidentes de Trabalho)
- Card azul 👥 → `/cipa/public` (Público Flutuante)

Detalhes visuais: grade responsiva (`grid-cols-1 sm:grid-cols-2`), efeito de hover no card (`hover:bg- ...`) e a setinha que desliza (`group` + `group-hover`).

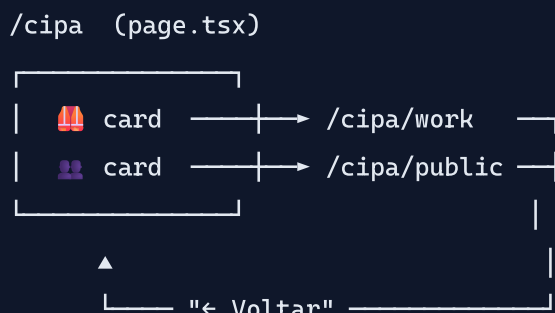
`cipa/work/page.tsx` e `cipa/public/page.tsx`

Server Components simples (por enquanto, placeholders "Em construção"). Cada uma tem um título e um `Link` "← Voltar para a CIPA" apontando para `/cipa`, fechando o ciclo de navegação.

`cipa/layout.tsx`

A moldura de toda a seção `/cipa`. Define o `metadata` (título da aba "CIPA | SmartCampus Mauá") e envolve o conteúdo num `<div>` que ocupa a altura da tela.

Fluxo de navegação



11. Detalhes do Next.js 16 (pegadinhas)

Coisas específicas desta versão, boas de saber:

- **Server Component é o padrão**; adicione `"use client"` para interatividade.
- `fetch` **não tem cache por padrão** — use `next: { revalidate: N }` ou `cache: "force-cache"` quando quiser reaproveitar respostas.
- `params` e `searchParams` **são "Promises"** — em páginas que recebem parâmetros da URL, é preciso usar `await` neles.
- O antigo **"middleware" virou "Proxy"** (`proxy.ts`) — interceptação de requisições, caso precisemos no futuro.

12. Glossário rápido

- **Componente** — função que devolve JSX (um pedaço de tela).
- **JSX** — sintaxe parecida com HTML, escrita dentro do código.
- **Rota / Route** — um endereço (URL) do site.
- **App Router** — o sistema do Next em que pastas viram rotas.
- **Server Component** — roda no servidor; entrega HTML pronto; sem interação.
- **Client Component** — `"use client"`; roda no navegador; permite interação.
- **Props** — os "parâmetros" que um componente recebe (ex.: `children`, `sensor`).
- `children` — conteúdo passado para dentro de um componente (usado em layouts).
- **Hook** — funções do React que começam com `use` (`useState`, `useEffect` ...).
- **Tailwind** — CSS por classes utilitárias prontas.
- **Monorepo** — vários projetos relacionados no mesmo repositório.
- **Bun** — runtime (executa o JavaScript/TypeScript); aqui substitui o Node.js.
- **CORS** — regra de segurança do navegador entre sites diferentes; evitada quando buscamos dados no servidor.

Próximos passos sugeridos (quando você quiser): adicionar um cabeçalho/menu fixo à seção CIPA via `layout.tsx`, e depois transformar `work` e `public` em páginas com conteúdo real (provavelmente já com formulários — aí entram os Client Components).